Object-Oriented Programming (OOP) - Complete Deep Dive

Let me break down every single concept from this lecture with extreme depth and clarity.

---

1. Traditional Programming — The Problem

What were we doing before OOP?

Before OOP, we wrote **procedural code** — just a series of instructions, one after another. To represent a student, you'd do this:

```
String name = "Aditya";
int age = 28;
int rollNumber = 1001;
String college = "IIT Guwahati";
```

Looks fine for one student. Now add a second:

```
String name2 = "Rohit";
int age2 = 28;
int rollNumber2 = 102;
String college2 = "IIT Guwahati";
```

Why is this a problem? (4 core problems)

Problem 1 — Too many independent variables to represent ONE entity

These 4 variables (`name`, `age`, `rollNumber`, `college`) are **completely independent** of each other in memory. The computer has no idea they belong together. You know they're related because *you* wrote them, but the code itself has zero awareness of that relationship. There's no block, no container, no grouping — just 4 loose variables floating around.

Problem 2 — Passing to functions is painful

Say you want a function to print a student's details:

```
void printStudent(String name, int age, int rollNumber, String college) {
    System.out.println(name + " " + age + " " + rollNumber + " " + college);
}
```

You must pass ALL 4 parameters every single time. If you forget even one, the function breaks or gives incomplete output. There's no concept of "pass me a student" — you have to pass each piece individually.

Problem 3 — New student = New set of variables

Every time you need another student, you need another full set of variables (`name3`, `age3`, `rollNumber3`...). This scales horribly. 100 students = 400 variables. Chaos.

Problem 4 — No authority / ownership over data

These 4 variables have no "owner." Anyone in the code can change any variable at any time. There's no concept of "this data belongs to THIS student." `name`, `age`, `rollNumber` — anyone can modify them, misuse them, or mix them up. There's no protection or organization.

---

2. What is Object-Oriented Programming?

It's a paradigm — a way of *thinking* about code

OOP is not just a feature. It's an entirely different **programming paradigm** — a completely different mindset for how you design and write code. Other paradigms exist too (like functional programming), but Java is built around OOP.

The core philosophy of OOP

OOP asks one fundamental question:

> **"Why do we write code?"**

Answer: To solve **real-world problems.**

And OOP says: *"If you're solving real-world problems, why not model your code to look like the real world?"*

This is called **mimicking the real world** in code. Instead of thinking in terms of loose variables and functions, you think in terms of **objects** — just like the real world is made of objects.

---

3. The Real World is Made of Objects

Everything is an object

In the real world, everything is an entity — a *thing* that exists. OOP calls all these things **objects**. Whether it's living (a student, an animal) or non-living (a chair, a car), everything is an object.

Objects have two things:

1. Characteristics (data/attributes) These are the *properties* that describe the object. A student has:

- Name
- Age
- Roll Number
- College

2. Behaviours (actions) These are the *things the object can do*. A student can:

- Mark attendance
- Submit assignment
- Take exam

This is the key insight OOP is built on: **real-world objects have both characteristics AND behaviours**, so our programming objects should too.

---

4. The Concept of Abstraction in Representation

"Student" is an abstract idea

When you say the word "Student," you're not referring to any specific person. You're describing a **template** — a concept of what a student *is*. Every student has a name, age, roll number, and college. But no specific values yet.

This abstract idea = **Class** in Java.

Specific students are concrete instances

"Arun is a student" — now we're talking about a specific, real student with actual values. This specific student = **Object** in Java.

You don't need to represent everything

A human being plays many roles — student, son/daughter, employee. Each role has its own characteristics. But when you're building a college portal, you only care about the **student** role. You don't care that he's also someone's son or a future employee.

This means: **you only include characteristics relevant to your problem.** This is a critical concept — OOP doesn't mean copying reality 100%. It means picking the relevant parts of reality and representing those.

---

5. Classes — The Blueprint

What is a Class?

A class is a **blueprint** or **template** that describes what an object will look like. It defines:

- What data (characteristics) the object will hold
- What methods (behaviours) the object can perform

It does NOT hold actual data itself. It just *describes the structure*.

Syntax:

```
class Student {
   String name;
   int age;
   int rollNumber;
   String college;
}
```

Let's break this down character by character:

- `class` — Java keyword that says "I'm defining a blueprint"
- `Student` — the name of the blueprint (starts with capital letter — naming convention)
- `{ }` — curly braces define the **code block** — everything inside belongs to this class
- `String name;` — a field (variable) that every Student object will have
- `int age;` — another field
- etc.

What has NOT happened yet?

When you write `class Student { ... }`, **no memory has been allocated**. Nothing exists in RAM yet. You've just written a definition — like an architect drawing house plans. The house doesn't exist yet.

---

6. User-Defined Data Types

Two types of data types in Java:

Primitive types (built into Java):

- `int`, `short`, `byte`, `long` (integers)
- `float`, `double` (decimals)
- `boolean` (true/false)
- `char` (single character)

Non-primitive types (objects):

- Arrays (already known)
- Classes you define — **User-Defined Data Types**

What makes it "user-defined"?

When you write `class Student`, you are **creating a new data type** called `Student`. Just like `int` is a data type that holds a number, `Student` is now a data type that holds name + age + rollNumber + college.

The difference: Java gave you `int`. *You* created `Student`. Hence: **user-defined**.

---

7. Creating Objects — The Deep Mechanics

Step 1: Declaration

Student s1;

This creates a **reference variable** called `s1` in the **Stack memory**. At this point:

- `s1` exists in stack
- `s1` currently holds `null` (nothing)

- No Student object exists anywhere yet

Step 2: Instantiation (creating the actual object)

s1 = new Student();

The `new` keyword does something very specific and important:

1. Goes to **Heap memory**
2. Allocates a chunk of space big enough to hold all of Student's fields (name, age, rollNumber, college)
3. Returns the **memory address** (e.g., `1001`) of where this chunk starts
4. That address gets stored inside `s1`

Both steps combined (most common way):

Student s1 = new Student();

The crucial distinction: s1 is NOT the object

This is one of the most misunderstood points in Java:

- `s1` is a **reference variable** — it lives in Stack memory and stores a **memory address**
- The **actual object** lives in **Heap memory** and has NO name
- `s1` just *points to* (references) the object

Think of it like this: `s1` is a piece of paper with an address written on it. The actual house (object) is at that address. The paper is not the house.

In interviews, you may be asked: *"Is s1 an object?"* — The correct answer is **No. s1 is a reference variable that holds the address of an object.**

---

8. Two Types of Memory — Stack and Heap

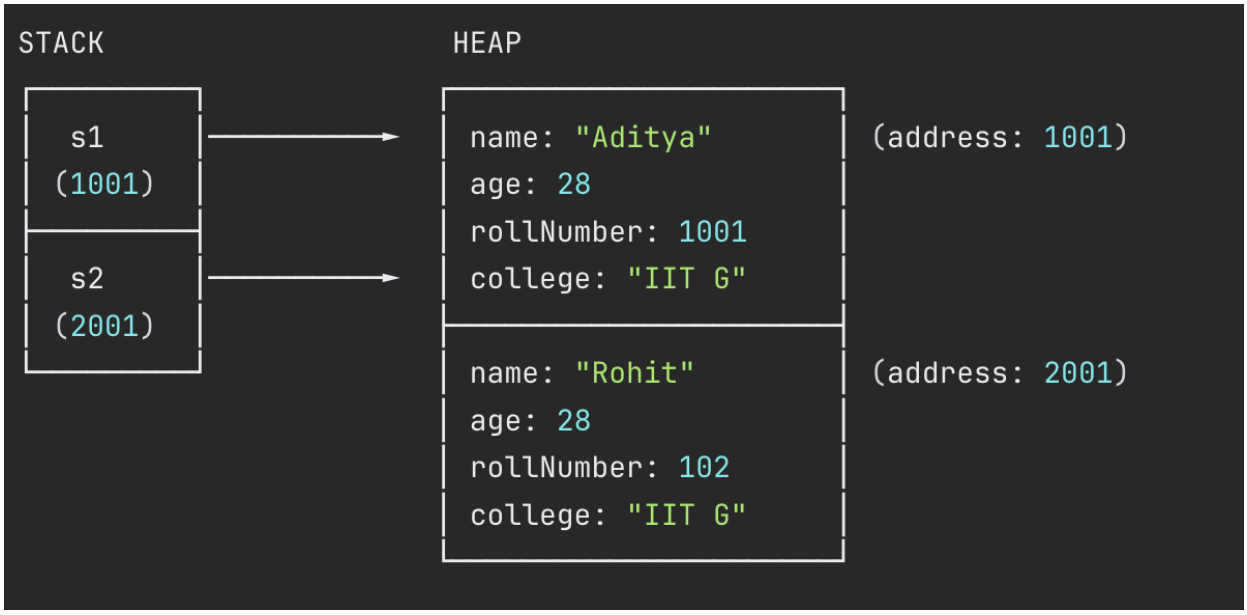
Stack Memory

- Stores **primitive variables** and **reference variables**
- Memory allocation happens at **compile time** (static memory allocation)
- Automatically managed — when a variable goes out of scope, it's removed
- Very fast access
- Small in size

Heap Memory

- Stores **actual objects**
- Memory allocation happens at **runtime** (dynamic memory allocation)
- Managed by Java's Garbage Collector
- Larger in size
- Slightly slower than stack

Visual representation:



---

9. Compile Time vs Runtime

Compile Time

- When Java **translates your source code** into bytecode
- Syntax errors are caught here (red underlines in IDE)
- Primitive variable memory allocation happens here
- Example: `int x = 4;` → memory for `x` is set at compile time

Runtime

- When the program is **actually executing**
- Object creation with `new` happens here
- This is called **Dynamic Memory Allocation** — memory is allocated *dynamically* as the program runs

- Errors that only appear when program runs are **runtime errors**

Why does this matter?

Because `new Student()` is a runtime operation, Java doesn't know how many Student objects you'll create until the program actually runs. You might create 0, or you might create 10,000. This flexibility is what makes `new` a "dynamic" operation.

	Compile Time	Runtime
Also called	Static	Dynamic
When	Before execution	During execution
Memory for	Primitives, references	Objects (via `new`)
Errors	Syntax errors	Logic/runtime errors

---

10. The Dot Operator — Your Bridge to the Object

What does `.` do?

The dot operator is how you **navigate from a reference variable to the actual object and its contents.**

Remember: `s1` just holds an address. The dot operator says: *"Go to that address, and give me access to what's inside."*

```
s1.name = "Aditya";    // Go to s1's object, access the 'name' field, set it to "Aditya"
s1.age = 28;           // Go to s1's object, access the 'age' field, set it to 28
System.out.println(s1.name);  // Go to s1's object, read the 'name' field
s1.markAttendance();   // Go to s1's object, execute the markAttendance method
```

Without the dot operator, you'd have no way to reach inside the heap object from the stack reference.

---

11. Characteristics vs Behaviours in Code

Characteristics → Fields (Variables)

These represent the **data** an object holds:

```
class Student {
    String name;        // characteristic
    int age;            // characteristic
    int rollNumber;     // characteristic
    String college;     // characteristic
}
```

Behaviours → Methods (Functions)

These represent **what the object can do**:

```
class Student {
    String name;
    int age;
    int rollNumber;
    String college;

    // BEHAVIOUR 1
    void markAttendance() {
        System.out.println("Attendance marked by " + name);
        // Notice: 'name' is directly accessible here — no parameter needed!
        // Because this method is INSIDE the class that owns 'name'
    }

    // BEHAVIOUR 2
    void printDetails() {
        System.out.println(name + " " + age + " " + rollNumber + " " + college);
    }
}
```

The superpower: methods inside the class can directly access fields

When `markAttendance()` uses `name`, it doesn't need `name` passed as a parameter. Why? Because the method is **inside the same class** that defines `name`. It has direct access. This is fundamentally different from the old procedural approach where you had to pass everything explicitly.

---

12. Naming Conventions in Java

These are **conventions** (good practices), NOT rules. The compiler won't stop you from breaking them, but every Java developer follows them.

Variables and Methods → camelCase

- First word: all lowercase
- Every subsequent word: first letter capitalized
- No spaces or underscores

int age; ✅
String firstName; ✅
int rollNumber; ✅
void markAttendance() {} ✅
void printStudentDetails(){}✅

int Age; ❌ (but won't cause error)
int first_name; ❌ (but won't cause error)

Classes → PascalCase (UpperCamelCase)

- Every word's first letter is capitalized
- Represents an **abstract concept**, hence the "elevated" naming

class Student {} ✅
class BankAccount {} ✅
class IITStudent {} ✅

class student {} ❌ (but won't cause error)
class bank_account {} ❌

Why does this matter in practice?

When you read someone's Java code, you can **instantly tell** what something is:

- Starts with lowercase → variable or method
- Starts with uppercase → class
- ALL_CAPS → constant (we'll see this later)

---

13. Non-Physical Objects — OOP Beyond the Real World

A critical extension of the OOP idea

We said OOP mimics the real world. But not everything has a physical form. Yet OOP can represent these too.

Examples of non-physical objects:

BankAccount:

```
class BankAccount {
   String accountNumber;
   String ifscCode;
   String branchName;
   double balance;

   void deposit(double amount) { ... }
   void withdraw(double amount) { ... }
}
```

A bank account doesn't physically exist. You can't touch it. But it has multiple characteristics and behaviours — so it DESERVES to be a class.

Location:

```
class Location {
   double latitude;
   double longitude;
   String name;
}
```

A location isn't a physical object. But it has multiple properties, so we represent it as a class.

The rule:

> If something has **multiple characteristics** that belong together, and/or **behaviours**, and can't be represented by a single primitive variable — make it a class.

---

14. The printDetails() Function — Why Put It Inside the Class?

Old way (procedural):

```
void printStudent(String name, int age, int rollNumber, String college) {
   System.out.println(name + age + rollNumber + college);
}
```

Problems:

- Must pass 4 separate parameters
- Must remember all 4 every time
- The function is "outside" — has no connection to the student

New way (OOP):

```
class Student {
   String name;
   int age;
   int rollNumber;
   String college;

   void printDetails() {
      // No parameters needed — we already have access to everything!
      System.out.println(name + " " + age + " " + rollNumber + " " + college);
   }
}
```

Now: `s1.printDetails();` — that's it. One call. No parameters. Clean.

But wait — printing isn't a "real" student behaviour!

True. A real human student can't print themselves. But the **digital** representation of a student can. OOP doesn't require you to *only* model real-world behaviours. You can add **digital behaviours** that make the code more useful. The philosophy is: model the real world as the foundation, then add what's needed for your program on top.

---

15. Understanding the Main Class

Why does Java need a class just to run?

Java is (almost) completely object-oriented. That means **everything must live inside a class.** You cannot write free-floating code outside a class in Java.

```
public class Demo {          // ← This is also a class!
   public static void main(String[] args) {  // ← Method inside the class
      // Your code here
      Student s1 = new Student();
      s1.name = "Aditya";
   }
}
```

Breaking down the boilerplate:

Part	Meaning
`public`	Access modifier (covered later)
`class Demo`	We're defining a class called Demo
`public static void main`	The entry point — JVM starts here
`String[] args`	Command-line arguments (covered later)
`static`	Keyword covered in future lecture

The key insight:

Even your "main" code lives inside a class (`Demo`). And your custom classes (`Student`) are separate. You create Student objects *inside* the main method of the Demo class. This is why Java is called an OOP language — you literally cannot escape classes.

Why "almost" completely OOP?

Java has **primitive types** (`int`, `char`, `boolean`, etc.) which are NOT objects. They're not instances of any class. A purely OOP language would make everything an object (like Smalltalk or Ruby). Java made primitives non-objects for **performance reasons**. Hence: *almost* completely OOP.

---

16. Complete Code — Putting It All Together

```
// Custom class — defined OUTSIDE the Demo class
class Student {
```

```
    // ── CHARACTERISTICS (Fields/Attributes) ──
    String name;
    int age;
    int rollNumber;
    String college;

    // ── BEHAVIOURS (Methods) ──

    void markAttendance() {
        // 'name' is directly accessible — no parameter needed
        System.out.println("Attendance marked by " + name);
    }

    void printDetails() {
        // All fields directly accessible
        System.out.println(name + ", " + age + ", " + rollNumber + ", " + college);
    }
}

// Main class — Java's entry point
public class Demo {
    public static void main(String[] args) {

        // Create object 1
        Student s1 = new Student();   // Heap allocates memory, s1 stores address
        s1.name = "Aditya";           // Dot operator navigates to object
        s1.age = 28;
        s1.rollNumber = 1001;
        s1.college = "IIT Guwahati";

        // Create object 2
        Student s2 = new Student();   // New separate memory block in Heap
        s2.name = "Rohit";
        s2.age = 28;
        s2.rollNumber = 102;
        s2.college = "IIT Guwahati";

        // Call behaviours
        s1.markAttendance();  // Output: Attendance marked by Aditya
        s2.markAttendance();  // Output: Attendance marked by Rohit

        s1.printDetails();    // Output: Aditya, 28, 1001, IIT Guwahati
        s2.printDetails();    // Output: Rohit, 28, 102, IIT Guwahati
    }
```

}

Summary: The Big Picture

REAL WORLD		JAVA CODE
Abstract idea (Student)	→	class Student { }
Characteristics	→	Fields (variables inside class)
Behaviours	→	Methods (functions inside class)
Specific person (Aditya)	→	Object (new Student())
"Aditya"	→	Reference variable s1

OOP's entire purpose is to make your code **mirror how you naturally think about the world** — in terms of things (objects) that have properties (fields) and can do things (methods). This makes large programs dramatically easier to write, read, debug, and maintain.

Everything Related to OOP That the Transcript Didn't Cover

---

1. The `null` Value — What Happens Before You Use `new`

When you declare a reference variable but don't assign it:

```
Student s1;  // declared but not initialized
```

`s1` holds `null`. Null means **"this reference points to nothing."** No object exists.

Now if you try:

```
System.out.println(s1.name);  // CRASH!
```

You get a **NullPointerException (NPE)** — the most famous error in Java. Why? Because you're asking the dot operator to navigate to an object, but there's no object to navigate to. The address stored in `s1` is null — it points to nothing.

```
// Safe way:
Student s1 = null;  // explicit null

if (s1 != null) {
    System.out.println(s1.name);  // only access if object exists
}
```

This is something every Java developer must deeply understand because NPE is the #1 runtime crash in Java applications.

---

2. Default Values of Fields — What's Inside a Fresh Object

When you do `new Student()` and haven't set any values yet, what do the fields contain? Java automatically sets **default values**:

```
Student s1 = new Student();

// What are the values RIGHT NOW, before you set anything?
System.out.println(s1.name);        // null   (String default)
System.out.println(s1.age);         // 0      (int default)
System.out.println(s1.rollNumber);  // 0      (int default)
System.out.println(s1.college);     // null   (String default)
```

Here's the full default value table:

Data Type	Default Value
`int`, `short`, `byte`, `long`	`0`
`float`, `double`	`0.0`
`boolean`	`false`

`char`	`'\u0000'` (null character)
Any object / String	`null`

This only applies to **fields inside a class**. Local variables (inside a method) do NOT get defaults — you must initialize them yourself or the compiler gives an error.

```
void someMethod() {
    int x;
    System.out.println(x);  // COMPILE ERROR — variable x might not be initialized
}
```

This distinction is very important and catches a lot of beginners off guard.

---

3. The `this` Keyword — The Object Referring to Itself

This is one of the most important concepts that wasn't mentioned. Inside any method of a class, `this` refers to **the current object that the method was called on.**

The problem it solves:

```
class Student {
    String name;
    int age;

    void setName(String name) {
        name = name;  // BUG! This assigns parameter to itself, not to the field!
    }
}
```

When the parameter name and the field name are the same, there's a **naming conflict.** Java gets confused. To resolve this:

```
class Student {
    String name;
    int age;

    void setName(String name) {
        this.name = name;
```

```
        // this.name = the field belonging to THIS object
        // name = the parameter passed in
    }
}
```

`this.name` explicitly means "the `name` field of this specific object."

`this` in depth:

```
class Student {
    String name;
    int age;

    void printDetails() {
        // These two lines do the EXACT same thing:
        System.out.println(name);
        System.out.println(this.name);  // 'this' is implicit when there's no conflict
    }

    void setData(String name, int age) {
        this.name = name;    // 'this' is REQUIRED here to disambiguate
        this.age = age;
    }

    Student getSelf() {
        return this;  // returning the current object itself
    }
}
```

Internally, when you call `s1.markAttendance()`, Java secretly passes `s1` as `this` to the method. So inside `markAttendance`, `this` = `s1`.

---

4. Constructors — The Proper Way to Initialize Objects

The transcript showed setting values like this:

```
Student s1 = new Student();
s1.name = "Aditya";
s1.age = 28;
s1.rollNumber = 1001;
```

```
s1.college = "IIT Guwahati";
```

This works but it's verbose and error-prone. What if you forget to set `rollNumber`? The object exists with `rollNumber = 0`, which is wrong. Enter **constructors.**

What is a constructor?

A constructor is a **special method that runs automatically when you create an object.** Its job is to initialize the object's fields the moment it's born.

```
class Student {
    String name;
    int age;
    int rollNumber;
    String college;

    // THIS IS A CONSTRUCTOR
    Student(String name, int age, int rollNumber, String college) {
        this.name = name;
        this.age = age;
        this.rollNumber = rollNumber;
        this.college = college;
    }
}
```

Rules of a constructor:

- **Same name as the class** (exactly)
- **No return type** — not even `void`
- Called automatically when `new` is used

Now creating an object becomes:

```
// All values set in ONE line, at the moment of creation
Student s1 = new Student("Aditya", 28, 1001, "IIT Guwahati");
Student s2 = new Student("Rohit", 25, 102, "IIT Guwahati");
```

Clean, safe, and impossible to forget a value.

Default Constructor

When you write NO constructor at all (like in the transcript), Java secretly provides a **default constructor** behind the scenes:

```
// What Java adds invisibly:
Student() {
    // empty — does nothing, fields get default values
}
```

That's why `new Student()` worked in the transcript without any constructor being written. The moment you write your own constructor, Java stops providing the default one.

Constructor Overloading

You can have multiple constructors with different parameters:

```
class Student {
    String name;
    int age;
    int rollNumber;
    String college;

    // Constructor 1 — no info, use defaults
    Student() {
        this.name = "Unknown";
        this.age = 0;
    }

    // Constructor 2 — just name and age
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Constructor 3 — full info
    Student(String name, int age, int rollNumber, String college) {
        this.name = name;
        this.age = age;
        this.rollNumber = rollNumber;
        this.college = college;
    }
}

// All three are valid:
Student s1 = new Student();
```

```
Student s2 = new Student("Aditya", 28);
Student s3 = new Student("Rohit", 25, 102, "IIT Guwahati");
```

Java picks the right constructor based on what arguments you pass. This is called **constructor overloading.**

---

5. The Four Pillars of OOP — The Big Picture

The transcript introduced the *idea* of OOP but didn't mention the four fundamental principles that define it. Everything in OOP flows from these:

Pillar 1: Encapsulation

"Bundle data and the methods that operate on that data together, and hide the internal details."

What the transcript showed was actually the beginning of encapsulation — putting fields and methods together in one class. But *true* encapsulation also means **hiding the fields** from direct outside access:

```
class Student {
    // Fields are PRIVATE — nobody outside can directly touch them
    private String name;
    private int age;

    // GETTERS — controlled read access
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    // SETTERS — controlled write access with validation
    public void setAge(int age) {
        if (age > 0 && age < 150) {  // validation!
            this.age = age;
        } else {
            System.out.println("Invalid age!");
        }
```

```
    }
}
```

Why does this matter? In the transcript, anyone could do:

```
s1.age = -500;  // This is WRONG but nothing stops it
```

With encapsulation, the setter protects the data:

```
s1.setAge(-500);  // Setter rejects this — "Invalid age!"
```

The object has **authority over its own data** — which directly solves Problem 4 from the transcript.

Pillar 2: Inheritance

"A class can inherit characteristics and behaviours from another class."

```
// Parent class
class Animal {
    String name;

    void eat() {
        System.out.println(name + " is eating");
    }

    void breathe() {
        System.out.println(name + " is breathing");
    }
}

// Child class — inherits everything from Animal
class Dog extends Animal {
    String breed;

    void bark() {
        System.out.println(name + " says: Woof!");
        // 'name' comes from Animal — Dog inherited it
    }
}

Dog d = new Dog();
```

```
d.name = "Tommy";
d.eat();    // Works! Inherited from Animal
d.breathe(); // Works! Inherited from Animal
d.bark();   // Dog's own method
```

Dog didn't rewrite `eat()` or `breathe()` — it got them for free from Animal. This avoids code duplication massively.

Pillar 3: Polymorphism

"One thing taking many forms."

The same method name behaves differently based on context:

```
class Animal {
    void makeSound() {
        System.out.println("Some generic sound");
    }
}

class Dog extends Animal {
    void makeSound() {  // Override — same name, different behaviour
        System.out.println("Woof!");
    }
}

class Cat extends Animal {
    void makeSound() {  // Override — same name, different behaviour
        System.out.println("Meow!");
    }
}

// Polymorphism in action:
Animal a1 = new Dog();
Animal a2 = new Cat();

a1.makeSound();  // Woof!  — Dog's version runs
a2.makeSound();  // Meow!  — Cat's version runs
```

Same method name `makeSound()`, called on the same type `Animal`, but different results based on actual object type.

Pillar 4: Abstraction

"Hide complexity, show only what's necessary."

You use a microwave without knowing how microwaves work internally. You press a button (interface), the microwave handles the complexity (implementation) internally.

```
abstract class Shape {
    // Force all shapes to implement this, but each does it differently
    abstract double calculateArea();

    void display() {
        System.out.println("Area: " + calculateArea());
    }
}

class Circle extends Shape {
    double radius;

    double calculateArea() {
        return 3.14159 * radius * radius;  // Circle's formula
    }
}

class Rectangle extends Shape {
    double width, height;

    double calculateArea() {
        return width * height;  // Rectangle's formula
    }
}
```

The user calls `calculateArea()` without knowing the formula. The complexity is hidden.

---

6. Access Modifiers — Controlling Visibility

The transcript mentioned `public` without explaining it. This is critical to understand alongside the class concept.

Modifier	Same Class	Same Package	Subclass	Everywhere
`private`	✅	❌	❌	❌
(default)	✅	✅	❌	❌
`protected`	✅	✅	✅	❌
`public`	✅	✅	✅	✅

```
class Student {
    public String name;      // Anyone can access
    private int age;         // Only Student class can access
    protected int rollNo;    // Student and its subclasses
    String college;          // Default — same package only
}
```

In the transcript's code, fields were written without any modifier — that's **package-private (default)**, not truly public. For good OOP, fields should be `private` with getter/setter methods.

---

7. Method Overloading — Same Name, Different Parameters

Different from constructor overloading but same concept — methods:

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {  // same name, different types
        return a + b;
    }

    int add(int a, int b, int c) {    // same name, different count
        return a + b + c;
    }
}
```

```
Calculator calc = new Calculator();
calc.add(2, 3);          // calls first method — returns 5
calc.add(2.5, 3.5);     // calls second method — returns 6.0
calc.add(1, 2, 3);      // calls third method — returns 6
```

Java decides which method to call based on:

1. Number of arguments
2. Types of arguments
3. Order of argument types

Return type alone is NOT enough to distinguish overloaded methods.

---

8. `static` Fields and Methods — Belonging to the Class, Not the Object

The transcript's boilerplate had `static` in `public static void main`. Here's what it actually means.

Non-static (instance) — belongs to each object:

```
class Student {
   String name;    // Each student has their OWN name
   int age;        // Each student has their OWN age
}

Student s1 = new Student();
Student s2 = new Student();

s1.name = "Aditya";
s2.name = "Rohit";
// s1.name and s2.name are DIFFERENT variables in DIFFERENT memory locations
```

Static — belongs to the CLASS itself, shared by all objects:

```
class Student {
   String name;
   int age;

   static int totalStudents = 0;  // ONE copy, shared by ALL students
```

```
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        totalStudents++;  // increment the shared counter
    }
}

Student s1 = new Student("Aditya", 28);
Student s2 = new Student("Rohit", 25);
Student s3 = new Student("Priya", 22);

System.out.println(Student.totalStudents);  // 3
// Notice: accessed via CLASS name, not object name
```

`totalStudents` is ONE variable in memory, not three. All three objects share it.

Why is `main` static?

```
public static void main(String[] args) { ... }
```

When Java starts your program, there are NO objects yet. Nothing has been created. But Java needs to call `main` to start. Since `main` is `static`, Java can call it directly on the class without needing to create an object first. That's the entire reason.

---

9. Object References — Assignment Behavior

This is deeply misunderstood and the transcript didn't cover it at all.

Primitives — value is copied:

```
int a = 5;
int b = a;  // a's VALUE (5) is copied into b
b = 10;     // changing b does NOT affect a

System.out.println(a);  // 5 — unchanged
System.out.println(b);  // 10
```

Objects — reference (address) is copied:

```
Student s1 = new Student("Aditya", 28);
Student s2 = s1;  // s1's ADDRESS is copied into s2
                  // NOT a new object — both point to same object!

s2.name = "Rohit";  // modifying through s2

System.out.println(s1.name);  // "Rohit" — s1 also changed!
System.out.println(s2.name);  // "Rohit"
```

Both `s1` and `s2` point to the **same object in heap**. Changing through one affects the other. This is one of the most common sources of bugs for beginners.

```
STACK          HEAP
s1 ──────────────► [name: "Rohit", age: 28]
s2 ──────────────────────────────────────────────^  (same address!)
```

To create a truly independent copy, you need to either use constructors or implement `clone()` — a more advanced topic.

---

10. Garbage Collection — What Happens to Abandoned Objects

When no reference variable points to an object anymore, that object becomes **unreachable.**

```
Student s1 = new Student("Aditya", 28);  // Object A created
s1 = new Student("Rohit", 25);           // Object B created, s1 now points to B
                                         // Object A has NO reference — it's abandoned!
```

Object A is now floating in heap memory with nothing pointing to it. Java's **Garbage Collector (GC)** automatically finds and destroys these abandoned objects, freeing memory.

You don't manually free memory in Java (unlike C/C++ where you must call `free()`). The GC runs in the background periodically. This is one of Java's biggest safety features — no memory leaks from forgotten manual cleanup.

---

11. Object as a Method Parameter — Pass by Reference Behavior

```
class Student {
   String name;
   int age;
}

class Main {

   static void birthday(Student s) {
      s.age = s.age + 1;  // modifying the object
   }

   public static void main(String[] args) {
      Student s1 = new Student();
      s1.name = "Aditya";
      s1.age = 28;

      birthday(s1);

      System.out.println(s1.age);  // 29 — the original object was modified!
   }
}
```

When you pass `s1` to `birthday()`, you're passing the **address** (reference). So `birthday()` modifies the **actual object in heap**, and those changes are visible back in `main`.

This is called **pass by reference behavior** (technically Java is always pass-by-value, but when it's an object, the value being passed is a reference/address — so the effect is reference-like).

---

12. `instanceof` Operator — Checking Object Type at Runtime

```
Student s1 = new Student("Aditya", 28);

System.out.println(s1 instanceof Student);  // true

// Useful when you have inheritance:
class Animal {}
```

```
class Dog extends Animal {}

Animal a = new Dog();
System.out.println(a instanceof Dog);     // true
System.out.println(a instanceof Animal);  // true — Dog IS-AN Animal
```

This operator lets you check what type an object actually is during runtime — very useful in polymorphism scenarios.

---

13. Object Comparison — `==` vs `.equals()`

Another extremely important concept:

`==` on objects compares ADDRESSES, not content:

```
Student s1 = new Student("Aditya", 28);
Student s2 = new Student("Aditya", 28);  // same data, different object

System.out.println(s1 == s2);  // FALSE — different memory addresses!
```

They have identical data but are two different objects in heap, so `==` returns false.

`.equals()` compares content:

```
String str1 = new String("Hello");
String str2 = new String("Hello");

System.out.println(str1 == str2);       // FALSE — different addresses
System.out.println(str1.equals(str2));  // TRUE — same content
```

For your custom classes, you need to **override** `equals()` to define what "equal" means for your objects — another advanced topic that builds directly on everything in the transcript.

---

14. The Full Object Lifecycle

From birth to death, here's what happens to every object:

1. CLASS LOADING
   JVM reads your Student.class file
   Blueprint is loaded into memory

2. OBJECT CREATION (new Student())
   Memory allocated in Heap
   Default values assigned to all fields
   Constructor runs
   Reference returned and stored in stack variable

3. OBJECT USAGE
   Accessed via reference variable + dot operator
   Fields read and written
   Methods called

4. OBJECT BECOMES UNREACHABLE
   Reference variable goes out of scope, OR
   Reference variable assigned to null, OR
   Reference variable assigned to different object

5. GARBAGE COLLECTION
   GC detects no references point to object
   Memory is freed and returned to heap pool

6. (Optional) finalize() method
   Called just before GC destroys the object
   Deprecated in modern Java, but historically used for cleanup

Understanding this full lifecycle helps you reason about memory usage and performance in Java programs.

Quick Reference Summary

Concept	What it is	Why it matters
`null`	Reference pointing to nothing	Causes NPE if accessed
Default values	Auto-assigned to fields	Prevents garbage data
`this`	Current object reference	Resolves naming conflicts

Constructor	Auto-runs on `new`	Proper initialization
Encapsulation	Bundle + hide data	Data protection
Inheritance	Child gets parent's stuff	Code reuse
Polymorphism	One name, many forms	Flexibility
Abstraction	Hide complexity	Simplicity
`static`	Belongs to class	Shared across objects
Reference copy	`s2 = s1` shares object	Common bug source
Garbage Collection	Auto memory cleanup	Memory safety
`==` vs `.equals()`	Address vs content	Correct comparison

Every single one of these builds directly on the foundation the transcript laid. Master the transcript's concepts first, then these will all click naturally.